State Management, Part 1

Debugging, Stateless vs Stateful, and where state should live

Dinu-Ștefan Rusu
FILS, Universitatea Politehnica București · Autumn 2026

TODAY

What you should leave with

Debug on purpose

Launch and attach DevTools, read the inspector, set breakpoints and step through Dart.

The three trees

Widget, Element and RenderObject, and what setState actually triggers.

Stateless vs Stateful

When a widget really needs a State object, and when it genuinely does not.

Sharing state

Lifting state up, InheritedWidget, and why value equality decides rebuilds.

PART 1 · DEBUGGING

Seeing what your app is actually doing

DevTools, breakpoints, and useful logging

The debugging toolbox

Hot reload

Injects changed source into the running VM and rebuilds. State objects survive.

Hot restart

Rebuilds from scratch and drops all state. Use it whenever initState or a global changed.

debugPrint & assert

Cheap, always available, and compiled out of release builds entirely.

Breakpoints

Stop on a line, inspect every variable in scope, step through the frame.

DevTools

Inspector, performance, CPU, memory, network: a browser tab attached to your app.

Read the error

Flutter's exception boxes name the widget and the constraint that failed.

print, debugPrint, and assert

```
print('value: $x');  // logcat truncates
debugPrint('value: $x'); // throttled
assert(count >= 0, 'count negative');
// Three switches worth memorizing:
debugDumpApp();          // widget tree
debugDumpRenderTree(); // sizes, constraints
debugPrintRebuildDirtyWidgets = true;
```

Why not just print?
Android's logcat silently drops lines beyond a length limit. `debugPrint` throttles output so nothing disappears.
`assert` takes a message. Use it for invariants you want to hear about in the lab and never in production.

Logging is a primary debugging tool. It is the only tool that works identically on a simulator, a real phone and a colleague's machine.

Launching and attaching DevTools

```
flutter run
#   DevTools debugger and profiler is available at:
#   http://127.0.0.1:9100?uri=http://127.0.0.1:53412/AbCdEfGh=/

dart devtools   # already running? paste that VM Service URI

# From the IDE, skip all of it:
#   VS Code:        Cmd/Ctrl+Shift+P -> "Dart: Open DevTools"
#   Android Studio: the "Flutter Inspector" tool window
```

Attach the tools before you need them. Performance work needs `flutter run --profile`: debug builds are JIT and assertion-heavy, so their timings mean nothing.

What each DevTools tab is for

Inspector

The live widget tree and the Layout Explorer for overflow.

Performance

Frame timeline, UI versus raster thread, which frames missed budget.

CPU Profiler

Where Dart time goes: a flame chart of your own functions.

Memory

Heap snapshots and allocation tracking. Find the controller you forgot to dispose.

Network

HTTP and WebSocket traffic, request by request. Back in Lecture 7.

Logging

The console stream, plus structured events and timeline entries.

The Widget Inspector: selecting and layout

Select Widget Mode: tap a pixel on the running device and the inspector jumps to that widget, and your IDE jumps to the source line that built it.

The Layout Explorer shows flex factors and incoming constraints. It is where you find the cause of a RenderFlex overflow, the yellow-and-black stripes.

You cannot reason about layout from the source alone. The Layout Explorer shows the constraints your widget actually received, not the ones you assumed.

The Widget Inspector: rebuilds and repaints

Track widget rebuilds

Counts how many times each widget rebuilt. That count is the measurement this lecture is about. Highlight repaints wraps every layer in a rotating color. Anything flashing constantly is repainting when it should not.

Try this in the lab

1. Run your lab app
2. Open the Inspector
3. Turn on Track widget rebuilds
4. Type one character into a field
5. Count what rebuilt

DEBUGGING

Why the frame budget matters

16.7 ms

per frame at 60 Hz

8.3 ms

per frame at 120 Hz, most 2026 phones

1

missed frame is visible jank

Frames, jank, and their causes

The performance overlay draws two bar charts: the UI thread, which runs your Dart build and layout, and the raster thread, where Impeller turns the layer tree into GPU commands. A bar that crosses the line is a dropped frame.

The usual causes sit squarely in this lecture: setState called too high in the tree, expensive work inside build, and a missing const.

Debug-mode timings are meaningless. Debug builds are JIT and run with assertions on. Measure jank only with `flutter run --profile`.

Breakpoints and stepping

Click the gutter in VS Code or Android Studio to set a breakpoint, then start the app with the debugger attached. Plain `flutter run` has no debugger.

Right-click a breakpoint to make it conditional, for example `todo.id == 42`. That beats a print inside a loop.

While paused, hover any variable, edit it in the Variables pane, and evaluate arbitrary Dart in the console.

1. Step over: run this line, do not descend into its calls
2. Step into: descend into the call on this line
3. Step out: finish this function, stop at the caller
4. Resume: run until the next breakpoint or exit

Hot reload keeps your State objects; hot restart throws them away. If a change does not appear, that is nearly always the reason.

PART 2 · STATE

What changes, and who owns it

Three trees, setState, and why Stateless still rebuilds

What counts as state

	EPHEMERAL: UI STATE	APP: SHARED STATE
Lives in	one widget's State object	above every widget that reads it
Looks like	a checkbox tick, a page index, text being typed	the signed-in user, theme mode, a cached feed
Disposed when	the widget leaves the tree	never, ideally: it outlives navigation
Reach for	setState	lift it up, then a shared holder

Most state is ephemeral. If setState in one widget is enough, that is the correct answer, not a beginner's answer.

Widgets are immutable blueprints

A widget is a description of a piece of UI, not the UI itself. Once constructed, its fields never change.

Changing the screen means constructing a new description, never editing the old one.

That is what makes the framework's job cheap: comparing two immutable descriptions is fast, and whole subtrees can be skipped.

const goes further.
The compiler canonicalizes a const widget, so the identical instance comes back every build.
`const Text('Ready')` is the same instance every build.
`Text('Ready')` is a new instance every build.

Immutability is what makes rebuilding cheap. The framework can prove nothing changed only because nothing can change in place.

The three trees

Every Flutter app maintains three parallel trees. Almost every rebuild question is answered by knowing which one you mean.

Widget

Immutable configuration, created and thrown away on every build. You write this.

Element

One per position in the tree. Holds your State object and decides: reuse it, or rebuild it.

RenderObject

Layout, paint and hit testing. Expensive to create, mutated in place whenever possible.

You build the left tree. The middle tree decides how little of the right tree has to change.

What setState actually does

Flutter does not rebuild the widget tree every frame.

setState marks exactly one Element dirty and requests a frame. Only the dirty subtrees run build again.

Sibling subtrees are never visited: their Elements, State objects and RenderObjects stay untouched.

1. `setState()`: you mutate State fields
2. `markNeedsBuild()`: Element is dirty
3. Next frame: build runs on dirty subtrees only
4. Reconcile children: runtimeType, then key
5. Layout and paint: only what changed

Only dirty subtrees rebuild. Push the setState call as far down the tree as you can.

Reuse: runtimeType, then key

```
// Same runtimeType AND same key: Element and State reused.
// Different runtimeType or key: subtree disposed, new one built.
ListView(
  children: [
    for (final todo in todos)
      TodoTile(key: ValueKey(todo.id), todo: todo),
  ],
);
// No key: Flutter matches children by position. Delete the first
// todo and the second row inherits the first row's state.
```

STATE

Stateless vs Stateful, precisely

	STATELESSWIDGET	STATEFULWIDGET
Mutable state	none of its own	a separate, long-lived State object
build() runs	on insert, and on every parent rebuild	all of that, plus on every setState
Lifecycle hooks	none	initState, didUpdateWidget, dispose
Genuinely fits	a label, an icon, a button that only calls a callback	a text field, an animation, a timer

A StatelessWidget does not ”build once”. It rebuilds whenever its parent does. It simply holds no mutable state of its own.

Inside a State object: the code

```
class _CounterFieldState extends State<CounterField> {
  final _controller = TextEditingController();
  int _count = 0;

  @override
  void dispose() {
    _controller.dispose();   // release what you created
    super.dispose();
  }
  void _increment() => setState(() => _count++);
  @override
  Widget build(BuildContext context) => Text('Count: $_count');
}
```

Inside a State object: the rules

State lives in the State class, never on the widget. Widget fields are re-created on every rebuild.

Read the widget's fields through `widget.something`; the framework refreshes that reference for you.

Anything you create, such as controllers, timers or stream subscriptions, you dispose.

Never call setState from build, and never after dispose: check `mounted` after an await.

PART 3 · SHARING STATE

When one widget is not enough

Lifting state up, InheritedWidget, and value equality

Lifting state up: the pattern

Two widgets need the same value, so it moves to their nearest common ancestor. Data flows down as constructor arguments; events flow back up as callbacks.

```
class _ParentState extends State<Parent> {
  double _target = 20;

  @override
  Widget build(BuildContext context) => Column(
    children: [
      TargetInput(onChanged: (v) => setState(() => _target = v)),
      TargetChart(target: _target),
    ],
  );
}
```

Lifting state up: the cost

The children stay reusable and know nothing about each other. That is the whole benefit.

Lift too far and you get prop drilling: a value threaded through five widgets that do not care about it, just to reach the sixth.

That is precisely the problem InheritedWidget solves.

Lifting state up is the default answer. Reach for a package only when the drilling becomes a real problem.

InheritedWidget: skip the drilling

```
class TargetScope extends InheritedWidget {
  const TargetScope({
    super.key,
    required this.target,
    required super.child,
  });
  final double target;
  static double of(BuildContext context) =>
      context.dependOnInheritedWidgetOfExactType<TargetScope>()!.target;

  @override
  bool updateShouldNotify(TargetScope old) => old.target != target;
}
```

Value equality: the Equatable pattern

```
class CounterState extends Equatable {
  const CounterState({required this.count, required this.status});
  final int count;
  final LoadStatus status;

  @override
  List<Object?> get props => [count, status];
}

const a = CounterState(count: 1, status: LoadStatus.ready);
const b = CounterState(count: 1, status: LoadStatus.ready);

a == b;   // true with Equatable, false without it
```

Value equality: why it matters

Every state consumer asks one question: is this state different from the last one?

Identity equality answers yes every time. Two structurally identical objects are still two objects, so the UI rebuilds when nothing changed.

The mirror failure: mutate a state object in place and re-emit the same instance, and identity answers no, so the UI does not rebuild even though the data did change.

State classes are immutable and compared by value. Add a field, add it to props. Forgetting is the classic bug.

Pick the smallest tool that works

TOOL	SCOPE	BOILERPLATE	TESTABLE	USE WHEN
setState	one widget	none	partly	the state never leaves the screen
Lifting up	a subtree	low	yes	two siblings share one value
InheritedWidget	any descendant	medium	yes	a value is read far below its owner
Provider	any descendant	low	yes	the same, with less ceremony
BLoC / Cubit	whole feature	high	excellent	events and side effects

Provider wraps InheritedWidget. Riverpod is its successor, without the BuildContext.

This is Part 1. setState, lifting up and InheritedWidget carry most screens. The last column is Lecture 6.

How Provider, Riverpod, and BLoC relate

Provider is a thin, ergonomic wrapper around InheritedWidget. It is not a different mechanism, just less code to write.

Riverpod is Provider's successor: the same idea without needing a BuildContext, and checked at compile time.

BLoC turns state into a stream of states driven by a stream of events. It costs a dependency and a pattern, in exchange for testability.

We name all three so you recognize them in a code review. Do not adopt a package you cannot explain. BLoC and Cubit are Lecture 6, and the graded lab app depends on them.

RECAP

Three things to remember

1. Widgets are immutable blueprints. The State object is what persists across builds.
2. setState marks one Element dirty. Only that subtree rebuilds on the next frame.
3. Value equality decides rebuilds. That is the whole reason Equatable exists.

RECAP

Further reading

docs.flutter.dev/tools/devtools

docs.flutter.dev/data-and-backend/state-mgmt/intro

api.flutter.dev: InheritedWidget, State, Key

pub.dev/packages/equatable

THIS WEEK

Run the lab app under DevTools, add ValueKey to a stateful list, and make one model extend Equatable.

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel